```
   int nProps = 0;
   foreach (var prop in props)
   {
      b.Text += String.Format("   {0} ({1})\n", prop.Name,
prop.PropertyType.Name);
      nProps++;
   }
   if (nProps == 0) b.Text += "   None\n";

   // Get methods.
   b.Text += "\nMethods:\n";
   var methods = t.GetTypeInfo().DeclaredMethods;
   int nMethods = 0;
   foreach (var method in methods)
   {
      if (method.IsSpecialName) continue;
      b.Text += String.Format("   {0}({1}) ({2})\n", method.Name,
                              GetSignature(method), method.ReturnType.Name);
      nMethods++;
   }
   if (nMethods == 0) b.Text += "   None\n";
}

private static string GetSignature(MethodInfo m)
{
   string signature = null;
   var parameters = m.GetParameters();
   for (int ctr = 0; ctr < parameters.Length; ctr++)
   {
      signature += String.Format("{0} {1}",
parameters[ctr].ParameterType.Name,
                                 parameters[ctr].Name);
      if (ctr < parameters.Length - 1) signature += ", ";
   }
   return signature;
}
```

Because metadata for the List<T> class isn't automatically included by the .NET Native tool chain, the example fails to display the requested member information at run time. To provide the necessary metadata, add the following `<Type>` element to the runtime directives file. Note that, because we've provided a parent <Namespace> element, we don't have to provide a fully qualified type name in the `<Type>` element.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
   <Application>
      <Assembly Name="*Application*" Dynamic="Required All" />
      <Namespace Name ="System.Collections.Generic" >
        <Type Name="List`1" Browse="Required All" />
     </Namespace>
```

```
    </Application>
</Directives>
```

Example 2

The following example uses reflection to retrieve a PropertyInfo object that represents the String.Chars[] property. It then uses the PropertyInfo.GetValue(Object, Object[]) method to retrieve the value of the seventh character in a string, and to display all the characters in the string. The variable `b` in the example is a TextBlock control.

C#

```
public void Example()
{
  string test = "abcdefghijklmnopqrstuvwxyz";

  // Get a PropertyInfo object.
  TypeInfo ti = typeof(string).GetTypeInfo();
  PropertyInfo pinfo = ti.GetDeclaredProperty("Chars");

  // Show the seventh letter ('g')
  object[] indexArgs = { 6 };
  object value = pinfo.GetValue(test, indexArgs);
  b.Text += String.Format("Character at position {0}: {1}\n", indexArgs[0],
value);

  // Show the complete string.
  b.Text += "\nThe complete string:\n";
  for (int x = 0; x < test.Length; x++)
  {
      b.Text += pinfo.GetValue(test, new Object[] {x}).ToString() + " ";
  }
}
// The example displays the following output:
//       Character at position 6: g
//
//       The complete string:
//       a b c d e f g h i j k l m n o p q r s t u v w x y z
```

Because metadata for the String object isn't available, the call to the PropertyInfo.GetValue(Object, Object[]) method throws a NullReferenceException exception at run time when compiled with the .NET Native tool chain. To eliminate the exception and provide the necessary metadata, add the following `<Type>` element to the runtime directives file:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="*Application*" Dynamic="Required All" />
    <Type Name="System.String" Dynamic="Required Public"/> -->
  </Application>
</Directives>
```

See also

- Runtime Directives (rd.xml) Configuration File Reference
- Runtime Directive Elements
- Runtime Directive Policy Settings

<TypeInstantiation> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to a constructed generic type.

Syntax

XML

```
<TypeInstantiation Name="type_name"
                   Arguments="type_arguments"
                   Activate="policy_type"
                   Browse="policy_type"
                   Dynamic="policy_type"
                   Serialize="policy_type"
                   DataContractSerializer="policy_setting"
                   DataContractJsonSerializer="policy_setting"
                   XmlSerializer="policy_setting"
                   MarshalObject="policy_setting"
                   MarshalDelegate="policy_setting"
                   MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
`Name`	General	Required attribute. Specifies the type name.
`Arguments`	General	Required attribute. Specifies the generic type arguments. If multiple arguments are present, they are separated by commas.
`Activate`	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
`Browse`	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.

Attribute	Attribute type	Description
`Dynamic`	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
`Serialize`	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
`DataContractSerializer`	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
`DataContractJsonSerializer`	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
`XmlSerializer`	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
`MarshalObject`	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
`MarshalDelegate`	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
`MarshalStructure`	Interop	Optional attribute. Controls policy for marshaling structures to native code.

Name attribute

Value	Description
type_name	The type name. If this `<TypeInstantiation>` element is the child of a <Namespace> element, a <Type> element, or another `<TypeInstantiation>` element, *type_name* can specify the name of the type without its namespace. Otherwise, *type_name* must include the fully qualified type name. The type name isn't decorated. For example, for a System.Collections.Generic.List<T> object, the `<TypeInstantiation>` element might appear as follows: `\<TypeInstantiation Name=System.Collections.Generic.List Dynamic="Required Public" />`

Arguments attribute

Value	Description
type_argument	Specifies the generic type arguments. If multiple arguments are present, they are separated by commas. Each argument must consist of the fully qualified type name.

All other attributes

Value	Description
policy_setting	The setting to apply to this policy type for the constructed generic type. Possible values are `All`, `Auto`, `Excluded`, `Public`, `PublicAndInternal`, `Required Public`, `Required PublicAndInternal`, and `Required All`. For more information, see Runtime Directive Policy Settings.

Child Elements

Element	Description
<Event>	Applies reflection policy to an event belonging to this type.
<Field>	Applies reflection policy to a field belonging to this type.
<ImpliesType>	Applies policy to a type, if that policy has been applied to the type represented by the containing `<TypeInstantiation>` element.
<Method>	Applies reflection policy to a method belonging to this type.
<MethodInstantiation>	Applies reflection policy to a constructed generic method belonging to this type.
<Property>	Applies reflection policy to a property belonging to this type.
<Type>	Applies reflection policy to a nested type.
`<TypeInstantiation>`	Applies reflection policy to a nested constructed generic type.

Parent Elements

Element	Description
<Application>	Serves as a container for application-wide types and type members whose metadata is available for reflection at run time.

Element	Description
<Assembly>	Applies reflection policy to all the types in a specified assembly.
<Library>	Defines the assembly that contains types and type members whose metadata is available for reflection at run time.
<Namespace>	Applies reflection policy to all the types in a namespace.
<Type>	Applies reflection policy to a type and all its members.
`<TypeInstantiation>`	Applies reflection policy to a constructed generic type and all its members.

Remarks

The reflection, serialization, and interop attributes are all optional. However, at least one must be present.

If a `<TypeInstantiation>` element is the child of an <Assembly>, <Namespace>, or <Type>, element, it overrides the policy settings defined by the parent element. If a <Type> element defines a corresponding generic type definition, the `<TypeInstantiation>` element overrides runtime reflection policy only for instantiations of the specified constructed generic type.

Example

The following example uses reflection to retrieve the generic type definition from a constructed Dictionary<TKey,TValue> object. It also uses reflection to display information about Type objects that represent constructed generic types and generic type definitions. The variable `b` in the example is a TextBlock control.

C#

```
public static void GetGenericInfo()
{
   // Get the type that represents the generic type definition and
   // display information about it.
   Type generic1 = typeof(Dictionary<,>);
   DisplayGenericType(generic1);

   // Get the type that represents a constructed generic type and its
   // generic type definition.
   Dictionary<string, Example> d1 = new Dictionary<string, Example>();
   Type constructed1 = d1.GetType();
   Type generic2 = constructed1.GetGenericTypeDefinition();
```

```
      // Display information for the generic type definition, and
      // for the constructed type Dictionary<String, Example>.
      DisplayGenericType(constructed1);
      DisplayGenericType(generic2);

      // Construct an array of type arguments.
      Type[] typeArgs = { typeof(string), typeof(Example) };
      // Construct the type Dictionary<String, Example>.
      Type constructed2 = generic1.MakeGenericType(typeArgs);

      DisplayGenericType(constructed2);

      object o = Activator.CreateInstance(constructed2);

      b.Text += "\r\nCompare types obtained by different methods:\n";
      b.Text += String.Format("   Are the constructed types equal? {0}\n",
                              (d1.GetType() == constructed2));
      b.Text += String.Format("   Are the generic definitions equal? {0}\n",
                              (generic1 ==
constructed2.GetGenericTypeDefinition()));

      // Demonstrate the DisplayGenericType and
      // DisplayGenericParameter methods with the Test class
      // defined above. This shows base, interface, and special
      // constraints.
      DisplayGenericType(typeof(TestGeneric<>));
   }

   // Display information about a generic type.
   private static void DisplayGenericType(Type t)
   {
      b.Text += String.Format("\n{0}\n", t);
      b.Text += String.Format("   Generic type? {0}\n",
                              t.GetTypeInfo().GenericTypeParameters.Length
!=
                              t.GenericTypeArguments.Length);
      b.Text += String.Format("   Generic type definition? {0}\n",
                              ! t.IsConstructedGenericType);

      // Get the generic type parameters.
      Type[] typeParameters = t.GetTypeInfo().GenericTypeParameters;
      if (typeParameters.Length > 0)
      {
         b.Text += String.Format("   {0} type parameters:\n",
                                 typeParameters.Length);
         foreach (Type tParam in typeParameters)
            b.Text += String.Format("      Type parameter: {0} position
{1}\n",
                    tParam.Name, tParam.GenericParameterPosition);
      }
      else
      {
         Type[] typeArgs = t.GenericTypeArguments;
         b.Text += String.Format("   {0} type arguments:\n",
                                 typeArgs.Length);
```

```
         foreach (var tArg in typeArgs)
               b.Text += String.Format("      Type argument: {0}\n",
                                       tArg);
      }
      b.Text += "\n-------------------------------\n";
   }
}

public interface ITestInterface { }

public class TestBase { }

public class TestGeneric<T> where T : TestBase, ITestInterface, new() { }

public class TestArgument : TestBase, ITestInterface
{
   public TestArgument()
   { }
}
```

After compilation with the .NET Native tool chain, the example throws a MissingMetadataException exception on the line that calls the Type.GetGenericTypeDefinition method. You can eliminate the exception and provide the necessary metadata by adding the following `<TypeInstantiation>` element to the runtime directives file:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="*Application*" Dynamic="Required All" />
     <TypeInstantiation Name="System.Collections.Generic.Dictionary"
                        Arguments="System.String,GenericType.Example"
                        Dynamic="Required Public" />
  </Application>
</Directives>
```

See also

- Runtime Directives (rd.xml) Configuration File Reference
- Runtime Directive Elements
- Runtime Directive Policy Settings

<TypeParameter> Element (.NET Native)

Article • 10/20/2022

Applies policy to the type represented by a Type argument passed to a method.

Syntax

XML

```
<Parameter Name="parameter_name"
           Activate="policy_type"
           Browse="policy_type"
           Dynamic="policy_type"
           Serialize="policy_type"
           DataContractSerializer="policy_type"
           DataContractJsonSerializer="policy_type"
           XmlSerializer="policy_type"
           MarshalObject="policy_type"
           MarshalDelegate="policy_type"
           MarshalStructure="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
`Name`	General	Required attribute. The name of the parameter of type Type. For example, for the method signature `Type.GetInterfaceMap(Type interfaceType)`, the value of the `Name` attribute is "interfaceType".
`Activate`	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
`Browse`	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
`Dynamic`	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
`Serialize`	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
`DataContractSerializer`	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
`DataContractJsonSerializer`	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
`XmlSerializer`	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
`MarshalObject`	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
`MarshalDelegate`	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
`MarshalStructure`	Interop	Optional attribute. Controls policy for marshaling value types to native code.

Name attribute

Value	Description
parameter_name	The name of the parameter of type Type. For example, for the method signature `Type.GetInterfaceMap(Type interfaceType)`, the value of the `Name` attribute is "interfaceType".

All other attributes

Value	Description
policy_setting	The setting to apply to this policy type. Possible values are `All`, `Public`, `PublicAndInternal`, `Required Public`, `Required PublicAndInternal`, and `Required All`. For more information, see Runtime Directive Policy Settings.

Child Elements

None.

Parent Elements

Element	Description
<Method>	Applies runtime reflection policy to a constructor or method.

Remarks

The `<TypeParameter>` element is similar to the <Parameter> element, except that it can be applied only to parameters of type Type. It applies policy to whatever type is represented at run time by the type argument specified by the `Name` attribute.

For example, the NewtonSoft JSON serializer includes a static `JsonConvert.DeserializeObject(String value, Type type)` method. The following reflection directives:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
   <Type Name="Newtonsoft.Json.JsonConvert" >
      <Method Name="DeserializeObject">
         <GenericParameter Name="type" Serialize="Required All" />
      </Method>
   </Type>
</Directives>
```

specify that metadata for the runtime type represented by the `type` argument should be made available for serialization. If these runtime directives apply to a project that includes the following source code:

C#

```
Type t = typeof(StockQuote);
Object obj = JsonConvert.DeserializeObject(data, t);
```

the reflection directives make metadata for the `StockQuote` type available for the NewtonSoft JSON serializer at run time.

See also

- <Method> Element
- Runtime Directives (rd.xml) Configuration File Reference
- Runtime Directive Policy Settings
- Runtime Directive Elements

Runtime Directive Policy Settings

Article • 10/20/2022

Runtime directive policy settings for .NET Native determine the availability of metadata for types and type members at run time. Without the necessary metadata, operations that rely on reflection, serialization and deserialization, or marshaling of .NET Framework types to COM or the Windows Runtime can fail and throw an exception. Most common exceptions are MissingMetadataException and (in the case of interop) MissingInteropDataException.

Runtime policy settings are controlled by a runtime directives (.rd.xml) file. Each runtime directive defines policy for a particular program element, such as an assembly (the <Assembly> element), a type (the <Type> element), or a method (the <Method> element). The directive includes one or more attributes that define the reflection policy types, the serialization policy types, and the interop policy types discussed in the next section. The value of the attribute defines the policy setting.

Policy types

Runtime directives files recognize three categories of policy types: reflection, serialization, and interop.

- Reflection policy types determine which metadata is made available at run time for reflection:
  - `Activate` controls runtime access to constructors, to enable activation of instances.
  - `Browse` controls querying for information about program elements.
  - `Dynamic` controls runtime access to all types and members to enable dynamic programming.

  The following table lists the reflection policy types and the program elements with which they can be used.

Element	Activate	Browse	Dynamic
<Application>	✔	✔	✔
<Assembly>	✔	✔	✔
<AttributeImplies>	✔	✔	✔
<Event>		✔	✔
<Field>		✔	✔
<GenericParameter>	✔	✔	✔

Element	Activate	Browse	Dynamic
<ImpliesType>	✔	✔	✔
<Method>		✔	✔
<MethodInstantiation>		✔	✔
<Namespace>	✔	✔	✔
<Parameter>	✔	✔	✔
<Property>		✔	✔
<Subtypes>	✔	✔	✔
<Type>	✔	✔	✔
<TypeInstantiation>	✔	✔	✔
<TypeParameter>	✔	✔	✔

- Serialization policy types determine which metadata is made available at run time for serialization and deserialization:
  - `Serialize` controls runtime access to constructors, fields, and properties, to enable type instances to be serialized by third-party libraries such as the Newtonsoft JSON serializer.
  - `DataContractSerializer` controls runtime access to constructors, fields, and properties, to enable type instances to be serialized by the DataContractSerializer class.
  - `DataContractJsonSerializer` controls runtime access to constructors, fields, and properties, to enable type instances to be serialized by the DataContractJsonSerializer class.
  - `XmlSerializer` controls runtime access to constructors, fields, and properties, to enable type instances to be serialized by the XmlSerializer class.

The following table lists the serialization policy types and the program elements with which they can be used.

Element	Serialize	DataContractSerializer	DataContractJsonSerializer	XmlSerializer
<Application>	✔	✔	✔	✔
<Assembly>	✔	✔	✔	✔
<AttributeImplies>	✔	✔	✔	✔
<Event>				
<Field>	✔			
<GenericParameter>	✔	✔	✔	✔

Element	Serialize	DataContractSerializer	DataContractJsonSerializer	XmlSerializer
<ImpliesType>	✔	✔	✔	✔
<Method>				
<MethodInstantiation>				
<Namespace>	✔	✔	✔	✔
<Parameter>	✔	✔	✔	✔
<Property>	✔			
<Subtypes>	✔	✔	✔	✔
<Type>	✔	✔	✔	✔
<TypeInstantiation>	✔	✔	✔	✔
<TypeParameter>	✔	✔	✔	✔

- Interop policy types determine which metadata is made available at run time to pass references types, value types, and function pointers to COM and the Windows Runtime:
  - `MarshalObject` controls native marshaling to COM and the Windows Runtime for reference types.
  - `MarshalDelegate` controls native marshaling of delegate types as function pointers.
  - `MarshalStructure` controls native marshaling to COM and the Windows Runtime for value types.

  The following table lists the interop policy types and the program elements with which they can be used.

Element	MarshalObject	MarshalDelegate	MarshalStructure
<Application>	✔	✔	✔
<Assembly>	✔	✔	✔
<AttributeImplies>	✔	✔	✔
<Event>			
<Field>			
<GenericParameter>	✔	✔	✔
<ImpliesType>	✔	✔	✔
<Method>			
<MethodInstantiation>			

Element	MarshalObject	MarshalDelegate	MarshalStructure
<Namespace>	✔	✔	✔
<Parameter>	✔	✔	✔
<Property>			
<Subtypes>	✔	✔	✔
<Type>	✔	✔	✔
<TypeInstantiation>	✔	✔	✔
<TypeParameter>	✔	✔	✔

Policy settings

Each policy type can be set to one of the values listed in the following table. Note that elements that represent type members support a different set of policy settings than other elements.

Policy setting	Description	`Assembly`, `Namespace`, `Type`, and `TypeInstantiation` elements	`Event`, `Field`, `Method`, `MethodInstantiation`, and `Property` elements
`All`	Enables the policy for all types and members that the .NET Native tool chain doesn't remove.	✔	
`Auto`	Specifies that the default policy should be used for the policy type for that program element. This is identical to omitting a policy for that policy type. `Auto` is typically used to indicate that policy is inherited from a parent element.	✔	✔
`Excluded`	Specifies that the policy is disabled for a particular program element. For example, the runtime directive: `<Type Name="BusinessClasses.Person" Browse="Excluded" Dynamic="Excluded" />` specifies that metadata for the `BusinessClasses.Person` class isn't available either to browse or to dynamically instantiate and modify `Person` objects.	✔	✔

Policy setting	Description	`Assembly`, `Namespace`, `Type`, and `TypeInstantiation` elements	`Event`, `Field`, `Method`, `MethodInstantiation`, and `Property` elements
`Included`	Enables a policy if metadata for the parent type is available.		✔
`Public`	Enables the policy for public types or members, unless the tool chain determines that the type or member is unnecessary and therefore removes it. This setting differs from `Required Public`, which ensures that metadata for public types and members is always available even if the tool chain determines that it's unnecessary.	✔	
`PublicAndInternal`	Enables the policy for public and internal types or members, unless the tool chain determines that the type or member is unnecessary and therefore removes it. This setting differs from `Required PublicAndInternal`, which ensures that metadata for public and internal types and members is always available even if the tool chain determines that it's unnecessary.	✔	
`Required`	Specifies that the policy for a member is enabled and that metadata will be available even if the member appears to be used.		✔
`Required Public`	Enables the policy for public types or members, and ensures that metadata for public types and members is always available. This setting differs from `Public`, which makes metadata for public types and members available only if the tool chain determines that it's necessary.	✔	
`Required PublicAndInternal`	Enables the policy for public and internal types or members, and ensures that metadata for public and internal types and members is always available. This setting differs from `PublicAndInternal`, which makes metadata for public and internal types and members available only if the tool chain determines that it's necessary.	✔	

Policy setting	Description	`Assembly`, `Namespace`, `Type`, and `TypeInstantiation` elements	`Event`, `Field`, `Method`, `MethodInstantiation`, and `Property` elements
`Required All`	Requires the tool chain to keep all types and members whether or not they're used, and enables policy for them.	✔	

See also

- Runtime Directives (rd.xml) Configuration File Reference
- Runtime Directive Elements

Serialization and metadata

Article • 10/20/2022

If your app serializes and deserializes objects, you may need to add entries to your runtime directives (.rd.xml) file to ensure that the necessary metadata is present at run time. There are two categories of serializers, and each requires different handling in your runtime directives file:

- Reflection-based third-party serializers. These require modifications to your runtime directives file, and are discussed in the next section.
- Non-reflection-based serializers found in the .NET Framework class library. These may require modifications to your runtime directives file, and are discussed in the Microsoft serializers section.

Third-party serializers

Third-party serializers, including Newtonsoft.JSON, typically are reflection-based. Given a binary large object (BLOB) of serialized data, the fields in the data are assigned to a concrete type by looking up the fields of the target type by name. At a minimum, using these libraries causes MissingMetadataException exceptions for each Type object that you try to serialize or deserialize in a `List<Type>` collection.

The easiest way to address issues caused by missing metadata for these serializers is to collect types that will be used in serialization under a single namespace (such as `App.Models`) and apply a `Serialize` metadata directive to it:

XML

```
<Namespace Name="App.Models" Serialize="Required PublicAndInternal" />
```

For information about the syntax used in the example, see <Namespace> Element.

Microsoft serializers

Although the DataContractSerializer, DataContractJsonSerializer, and XmlSerializer classes do not rely on reflection, they do require code to be generated based on the object to be serialized or deserialized. The overloaded constructors for each serializer include a Type parameter that specifies the type to be serialized or deserialized. How

you specify that type in your code defines the action you must take, as discussed in the next two sections.

typeof used in the constructor

If you call a constructor of these serialization classes and include the C# typeof operator in the method call, **you do not have to do any additional work**. For example, in each of the following calls to a serialization class constructor, the `typeof` keyword is used as part of the expression passed to the constructor.

C#

```
XmlSerializer xmlSer = new XmlSerializer(typeof(T));
DataContractSerializer dataSer = new DataContractSerializer(typeof(T));
DataContractJsonSerializer jsonSer = new
DataContractJsonSerializer(typeof(T));
```

The .NET Native compiler will automatically handle this code.

typeof used outside the constructor

If you call a constructor of these serialization classes and use the C# typeof operator outside the expression supplied to the constructor's Type parameter, as in the following code, the .NET Native compiler cannot resolve the type:

C#

```
Type t = typeof(DataSet);
XmlSerializer ser = new XmlSerializer(t);
```

In this case, you must specify the type in the runtime directives file by adding an entry like this:

XML

```
<Type Name="DataSet" Browse="Required Public" />
```

Similarly, if you call a constructor such as XmlSerializer(Type, Type[]) and provide an array of additional Type objects to serialize, as in the following code, the .NET Native compiler cannot resolve these types.

C#

```
XmlSerializer xSerializer = new XmlSerializer(typeof(Teacher),
                            new Type[] { typeof(Student),
                                         typeof(Course),
                                         typeof(Location) });
```

Add entries such as the following for each type to the runtime directives file:

XML

```
<Type Name="t" Browse="Required Public" />
```

For information about the syntax used in the example, see <Type> Element.

See also

- Runtime Directives (rd.xml) Configuration File Reference
- Runtime Directive Elements
- <Type> Element
- <Namespace> Element

Migrate Your Windows 8.x App to .NET Native

Article • 10/20/2022

.NET Native provides static compilation of apps in the Microsoft Store or on the developer's computer. This differs from the dynamic compilation performed for Windows 8.x apps (also previously called Microsoft Store apps) by the just-in-time (JIT) compiler or the Native Image Generator (Ngen.exe) on the device. Despite the differences, .NET Native tries to maintain compatibility with .NET for Windows 8.x apps. For the most part, things that work on the .NET for Windows 8.x apps also work with .NET Native. However, in some cases, you may encounter behavioral changes. This document discusses these differences between the standard .NET for Windows 8.x apps and .NET Native in the following areas:

- General runtime differences
- Dynamic programming differences
- Other reflection-related differences
- Unsupported scenarios and APIs
- Visual Studio differences

General runtime differences

- Exceptions, such as TypeLoadException, that are thrown by the JIT compiler when an app runs on the common language runtime (CLR) generally result in compile-time errors when processed by .NET Native.
- Don't call the GC.WaitForPendingFinalizers method from an app's UI thread. This can result in a deadlock on .NET Native.
- Don't rely on static class constructor invocation ordering. In .NET Native, the invocation order is different from the order on the standard runtime. (Even with the standard runtime, you shouldn't rely on the order of execution of static class constructors.)
- Infinite looping without making a call (for example, `while(true);`) on any thread may bring the app to a halt. Similarly, large or infinite waits may bring the app to a halt.

- Certain generic initialization cycles don't throw exceptions in .NET Native. For example, the following code throws a TypeLoadException exception on the standard CLR. In .NET Native, it doesn't.

```
C#
```

```
using System;

struct N<T> {}
struct X { N<X> x; }

public class Example
{
   public static void Main()
   {
      N<int> n = new N<int>();
      X x = new X();
   }
}
```

- In some cases, .NET Native provides different implementations of .NET Framework class libraries. An object returned from a method will always implement the members of the returned type. However, since its backing implementation is different, you may not be able to cast it to the same set of types as you could on other .NET Framework platforms. For example, in some cases, you may not be able to cast the IEnumerable<T> interface object returned by methods such as TypeInfo.DeclaredMembers or TypeInfo.DeclaredProperties to `T[]`.

- The WinInet cache isn't enabled by default on .NET for Windows 8.x apps, but it is on .NET Native. This improves performance but has working set implications. No developer action is necessary.

Dynamic programming differences

.NET Native statically links in code from .NET Framework to make the code app-local for maximum performance. However, binary sizes have to remain small, so the entire .NET Framework can't be brought in. The .NET Native compiler resolves this limitation by using a dependency reducer that removes references to unused code. However, .NET Native might not maintain or generate some type information and code when that information can't be inferred statically at compile time, but instead is retrieved dynamically at runtime.

.NET Native does enable reflection and dynamic programming. However, not all types can be marked for reflection, because this would make the generated code size too

large (especially because reflecting on public APIs in .NET Framework is supported). The .NET Native compiler makes smart choices about which types should support reflection, and it keeps the metadata and generates code only for those types.

For example, data binding requires an app to be able to map property names to functions. In .NET for Windows 8.x apps, the common language runtime automatically uses reflection to provide this capability for managed types and publicly available native types. In .NET Native, the compiler automatically includes metadata for types to which you bind data.

The .NET Native compiler can also handle commonly used generic types such as List<T> and Dictionary<TKey,TValue>, which work without requiring any hints or directives. The dynamic keyword is also supported within certain limits.

> **Note**
>
> You should test all dynamic code paths thoroughly when porting your app to .NET Native.

The default configuration for .NET Native is sufficient for most developers, but some developers might want to fine- tune their configurations by using a runtime directives (.rd.xml) file. In addition, in some cases, the .NET Native compiler is unable to determine which metadata must be available for reflection and relies on hints, particularly in the following cases:

- Some constructs like Type.MakeGenericType and MethodInfo.MakeGenericMethod can't be determined statically.
- Because the compiler can't determine the instantiations, a generic type that you want to reflect on has to be specified by runtime directives. This isn't just because all code must be included, but because reflection on generic types can form an infinite cycle (for example, when a generic method is invoked on a generic type).

> **Note**
>
> Runtime directives are defined in a runtime directives (.rd.xml) file. For general information about using this file, see **Getting Started**. For information about the runtime directives, see **Runtime Directives (rd.xml) Configuration File Reference**.

.NET Native also includes profiling tools that help the developer determine which types outside the default set should support reflection.

Other reflection-related differences

There are a number of other individual reflection-related differences in behavior between .NET for Windows 8.x apps and .NET Native.

In .NET Native:

- Private reflection over types and members in the .NET Framework class library is not supported. You can, however, reflect over your own private types and members, as well as types and members in third-party libraries.
- The ParameterInfo.HasDefaultValue property correctly returns `false` for a ParameterInfo object that represents a return value. In .NET for Windows 8.x apps, it returns `true`. Intermediate language (IL) doesn't support this directly, and interpretation is left to the language.
- Public members on the RuntimeFieldHandle and RuntimeMethodHandle structures aren't supported. These types are supported only for LINQ, expression trees, and static array initialization.
- RuntimeReflectionExtensions.GetRuntimeProperties and RuntimeReflectionExtensions.GetRuntimeEvents include hidden members in base classes and thus may be overridden without explicit overrides. This is also true of other RuntimeReflectionExtensions.GetRuntime* methods.
- Type.MakeArrayType and Type.MakeByRefType don't fail when you try to create certain combinations (for example, an array of `byref` objects).
- You can't use reflection to invoke members that have pointer parameters.
- You can't use reflection to get or set a pointer field.
- When the argument count is wrong and the type of one of the arguments is incorrect, .NET Native throws an ArgumentException instead of a TargetParameterCountException.
- Binary serialization of exceptions is generally not supported. As a result, non-serializable objects can be added to the Exception.Data dictionary.

Unsupported scenarios and APIs

The following sections list unsupported scenarios and APIs for general development, interop, and technologies such as HTTPClient and Windows Communication Foundation (WCF):

- General development
- HttpClient
- Interop
- Unsupported APIs

General development differences

Value types

- If you override the ValueType.Equals and ValueType.GetHashCode methods for a value type, don't call the base class implementations. In .NET for Windows 8.x apps, these methods rely on reflection. At compile time, .NET Native generates an implementation that doesn't rely on runtime reflection. This means that if you don't override these two methods, they will work as expected, because .NET Native generates the implementation at compile time. However, overriding these methods but calling the base class implementation results in an exception.
- Value types larger than 1 megabyte aren't supported.
- Value types can't have a parameterless constructor in .NET Native. (C# and Visual Basic prohibit parameterless constructors on value types. However, these can be created in IL.)

Arrays

- Arrays with a lower bound other than zero aren't supported. Typically, these arrays are created by calling the Array.CreateInstance(Type, Int32[], Int32[]) overload.
- Dynamic creation of multidimensional arrays isn't supported. Such arrays are typically created by calling an overload of the Array.CreateInstance method that includes a `lengths` parameter, or by calling the Type.MakeArrayType(Int32) method.
- Multidimensional arrays that have four or more dimensions aren't supported; that is, their Array.Rank property value is four or greater. Use jagged arrays (an array of arrays) instead. For example, `array[x,y,z]` is invalid, but `array[x][y][z]` isn't.
- Variance for multidimensional arrays isn't supported and causes an InvalidCastException exception at run time.

Generics

- Infinite generic type expansion results in a compiler error. For example, this code fails to compile:

```
C#

class A<T> {}

class B<T> : A<B<A<T>>>
{}
```

Pointers

- Arrays of pointers aren't supported.
- You can't use reflection to get or set a pointer field.

Serialization

The KnownTypeAttribute(String) attribute isn't supported. Use the KnownTypeAttribute(Type) attribute instead.

Resources

The use of localized resources with the EventSource class isn't supported. The EventSourceAttribute.LocalizationResources property doesn't define localized resources.

Delegates

`Delegate.BeginInvoke` and `Delegate.EndInvoke` aren't supported.

Miscellaneous APIs

- The TypeInfo.GUID property throws a PlatformNotSupportedException exception if a GuidAttribute attribute isn't applied to the type. The GUID is used primarily for COM support.
- The DateTime.Parse method correctly parses strings that contain short dates in .NET Native. However, it doesn't maintain compatibility with certain changes in date and time parsing.
- BigInteger.ToString `("E")` is correctly rounded in .NET Native. In some versions of the CLR, the result string is truncated instead of rounded.

HttpClient differences

In .NET Native, the HttpClientHandler class internally uses WinINet (through the HttpBaseProtocolFilter class) instead of the WebRequest and WebResponse classes used in the standard .NET for Windows 8.x apps. WinINet doesn't support all the configuration options that the HttpClientHandler class supports. As a result:

- Some of the capability properties on HttpClientHandler return `false` on .NET Native, whereas they return `true` in the standard .NET for Windows 8.x apps.
- Some of the configuration property `get` accessors always return a fixed value on .NET Native that is different than the default configurable value in .NET for Windows 8.x apps.

Some additional behavior differences are covered in the following subsections.

Proxy

The HttpBaseProtocolFilter class doesn't support configuring or overriding the proxy on a per-request basis. This means that all requests on .NET Native use the system-configured proxy server or no proxy server, depending on the value of the HttpClientHandler.UseProxy property. In .NET for Windows 8.x apps, the proxy server is defined by the HttpClientHandler.Proxy property. On .NET Native, setting the HttpClientHandler.Proxy to a value other than `null` throws a PlatformNotSupportedException exception. The HttpClientHandler.SupportsProxy property returns `false` on .NET Native, whereas it returns `true` in the standard .NET Framework for Windows 8.x apps.

Automatic redirection

The HttpBaseProtocolFilter class doesn't allow the maximum number of automatic redirections to be configured. The value of the HttpClientHandler.MaxAutomaticRedirections property is 50 by default in the standard .NET for Windows 8.x apps and can be modified. On .NET Native, the value of this property is 10, and trying to modify it throws a PlatformNotSupportedException exception. The HttpClientHandler.SupportsRedirectConfiguration property returns `false` on .NET Native, whereas it returns `true` in .NET for Windows 8.x apps.

Automatic decompression

.NET for Windows 8.x apps allows you to set the HttpClientHandler.AutomaticDecompression property to Deflate, GZip, both Deflate and GZip, or None. .NET Native only supports Deflate together with GZip, or None. Trying to set the AutomaticDecompression property to either Deflate or GZip alone silently sets it to both Deflate and GZip.

Cookies

Cookie handling is performed simultaneously by HttpClient and WinINet. Cookies from the CookieContainer are combined with cookies in the WinINet cookie cache. Removing a cookie from CookieContainer prevents HttpClient from sending the cookie, but if the cookie was already seen by WinINet, and cookies weren't deleted by the user, WinINet sends it. It isn't possible to programmatically remove a cookie from WinINet by using the HttpClient, HttpClientHandler, or CookieContainer API. Setting the HttpClientHandler.UseCookies property to `false` causes only HttpClient to stop sending cookies; WinINet might still include its cookies in the request.

Credentials

In .NET for Windows 8.x apps, the HttpClientHandler.UseDefaultCredentials and HttpClientHandler.Credentials properties work independently. Additionally, the Credentials property accepts any object that implements the ICredentials interface. In .NET Native, setting the UseDefaultCredentials property to `true` causes the Credentials property to become `null`. In addition, the Credentials property can be set only to `null`, DefaultCredentials, or an object of type NetworkCredential. Assigning any other ICredentials object, the most popular of which is CredentialCache, to the Credentials property throws a PlatformNotSupportedException.

Other unsupported or unconfigurable features

In .NET Native:

- The value of the HttpClientHandler.ClientCertificateOptions property is always Automatic. In .NET for Windows 8.x apps, the default is Manual.
- The HttpClientHandler.MaxRequestContentBufferSize property isn't configurable.
- The HttpClientHandler.PreAuthenticate property is always `true`. In .NET for Windows 8.x apps, the default is `false`.
- The `SetCookie2` header in responses is ignored as obsolete.

Interop differences

Deprecated APIs

A number of infrequently used APIs for interoperability with managed code have been deprecated. When used with .NET Native, these APIs may throw a NotImplementedException or PlatformNotSupportedException exception, or result in a

compiler error. In .NET for Windows 8.x apps, these APIs are marked as obsolete, although calling them generates a compiler warning rather than a compiler error.

Deprecated APIs for `VARIANT` marshaling include:

- System.Runtime.InteropServices.BStrWrapper
- System.Runtime.InteropServices.CurrencyWrapper
- System.Runtime.InteropServices.DispatchWrapper
- System.Runtime.InteropServices.ErrorWrapper
- System.Runtime.InteropServices.UnknownWrapper
- System.Runtime.InteropServices.VariantWrapper
- UnmanagedType.IDispatch
- UnmanagedType.SafeArray
- System.Runtime.InteropServices.VarEnum

UnmanagedType.Struct is supported, but it throws an exception in some scenarios, such as when it is used with IDispatch or `byref` variants.

Deprecated APIs for IDispatch support include:

- System.Runtime.InteropServices.ClassInterfaceType.AutoDispatch
- System.Runtime.InteropServices.ClassInterfaceType.AutoDual
- System.Runtime.InteropServices.ComDefaultInterfaceAttribute

Deprecated APIs for classic COM events include:

- System.Runtime.InteropServices.ComEventsHelper
- ComSourceInterfacesAttribute

Deprecated APIs in the System.Runtime.InteropServices.ICustomQueryInterface interface, which isn't supported in .NET Native, include:

- System.Runtime.InteropServices.ICustomQueryInterface (all members)
- System.Runtime.InteropServices.CustomQueryInterfaceMode (all members)
- System.Runtime.InteropServices.CustomQueryInterfaceResult (all members)
- System.Runtime.InteropServices.Marshal.GetComInterfaceForObject(Object, Type, CustomQueryInterfaceMode)

Other unsupported interop features include:

- System.Runtime.InteropServices.ICustomAdapter (all members)
- System.Runtime.InteropServices.SafeBuffer (all members)
- System.Runtime.InteropServices.UnmanagedType.Currency
- System.Runtime.InteropServices.UnmanagedType.VBByRefStr

- System.Runtime.InteropServices.UnmanagedType.AnsiBStr
- System.Runtime.InteropServices.UnmanagedType.AsAny
- System.Runtime.InteropServices.UnmanagedType.CustomMarshaler

Rarely used marshaling APIs:

- System.Runtime.InteropServices.Marshal.ReadByte(Object, Int32)
- System.Runtime.InteropServices.Marshal.ReadInt16(Object, Int32)
- System.Runtime.InteropServices.Marshal.ReadInt32(Object, Int32)
- System.Runtime.InteropServices.Marshal.ReadInt64(Object, Int32)
- System.Runtime.InteropServices.Marshal.ReadIntPtr(Object, Int32)
- System.Runtime.InteropServices.Marshal.WriteByte(Object, Int32, Byte)
- System.Runtime.InteropServices.Marshal.WriteInt16(Object, Int32, Int16)
- System.Runtime.InteropServices.Marshal.WriteInt32(Object, Int32, Int32)
- System.Runtime.InteropServices.Marshal.WriteInt64(Object, Int32, Int64)
- System.Runtime.InteropServices.Marshal.WriteIntPtr(Object, Int32, IntPtr)

Platform invoke and COM interop compatibility

Most platform invoke and COM interop scenarios are still supported in .NET Native. In particular, all interoperability with Windows Runtime (WinRT) APIs and all marshaling required for the Windows Runtime is supported. This includes marshaling support for:

- Arrays (including UnmanagedType.ByValArray)
- `BStr`
- Delegates
- Strings (Unicode, ANSI, and HSTRING)
- Structs (`byref` and `byval`)
- Unions
- Win32 handles
- All WinRT constructs
- Partial support for marshaling variant types. The following are supported:
  - Boolean
  - Byte
  - Decimal

    - Double
    - Int16
    - Int32
    - Int64
    - SByte
    - Single
    - UInt16
    - UInt32
    - UInt64
    - `BStr`
    - IUnknown

However, .NET Native doesn't support the following:

- Using classic COM events
- Implementing the System.Runtime.InteropServices.ICustomQueryInterface interface on a managed type
- Implementing the IDispatch interface on a managed type through the System.Runtime.InteropServices.ComDefaultInterfaceAttribute attribute. However, you can't call COM objects through `IDispatch`, and your managed object can't implement `IDispatch`.

Using reflection to invoke a platform invoke method isn't supported. You can work around this limitation by wrapping the method call in another method and using reflection to call the wrapper instead.

Other differences from .NET APIs for Windows 8.x apps

This section lists the remaining APIs that aren't supported in .NET Native. The largest set of the unsupported APIs is the Windows Communication Foundation (WCF) APIs.

DataAnnotations (System.ComponentModel.DataAnnotations)

The types in the System.ComponentModel.DataAnnotations and System.ComponentModel.DataAnnotations.Schema namespaces aren't supported in

.NET Native. These include the following types that are present in .NET for Windows 8.x apps:

- System.ComponentModel.DataAnnotations.AssociationAttribute
- System.ComponentModel.DataAnnotations.ConcurrencyCheckAttribute
- System.ComponentModel.DataAnnotations.CustomValidationAttribute
- System.ComponentModel.DataAnnotations.DataType
- System.ComponentModel.DataAnnotations.DataTypeAttribute
- System.ComponentModel.DataAnnotations.DisplayAttribute
- System.ComponentModel.DataAnnotations.DisplayColumnAttribute
- DisplayFormatAttribute
- EditableAttribute
- EnumDataTypeAttribute
- System.ComponentModel.DataAnnotations.FilterUIHintAttribute
- System.ComponentModel.DataAnnotations.KeyAttribute
- System.ComponentModel.DataAnnotations.RangeAttribute
- System.ComponentModel.DataAnnotations.RegularExpressionAttribute
- System.ComponentModel.DataAnnotations.RequiredAttribute
- System.ComponentModel.DataAnnotations.StringLengthAttribute
- System.ComponentModel.DataAnnotations.TimestampAttribute
- System.ComponentModel.DataAnnotations.UIHintAttribute
- System.ComponentModel.DataAnnotations.ValidationAttribute
- System.ComponentModel.DataAnnotations.ValidationContext
- System.ComponentModel.DataAnnotations.ValidationException
- System.ComponentModel.DataAnnotations.ValidationResult
- System.ComponentModel.DataAnnotations.Validator
- System.ComponentModel.DataAnnotations.Schema.DatabaseGeneratedAttribute
- System.ComponentModel.DataAnnotations.Schema.DatabaseGeneratedOption

Visual Basic

Visual Basic isn't currently supported in .NET Native. The following types in the Microsoft.VisualBasic and Microsoft.VisualBasic.CompilerServices namespaces aren't available in .NET Native:

- Microsoft.VisualBasic.CallType
- Microsoft.VisualBasic.Constants
- Microsoft.VisualBasic.HideModuleNameAttribute
- Microsoft.VisualBasic.Strings
- Microsoft.VisualBasic.CompilerServices.Conversions
- Microsoft.VisualBasic.CompilerServices.DesignerGeneratedAttribute
- Microsoft.VisualBasic.CompilerServices.IncompleteInitialization

- Microsoft.VisualBasic.CompilerServices.NewLateBinding
- Microsoft.VisualBasic.CompilerServices.ObjectFlowControl
- Microsoft.VisualBasic.CompilerServices.ObjectFlowControl.ForLoopControl
- Microsoft.VisualBasic.CompilerServices.Operators
- Microsoft.VisualBasic.CompilerServices.OptionCompareAttribute
- Microsoft.VisualBasic.CompilerServices.OptionTextAttribute
- Microsoft.VisualBasic.CompilerServices.ProjectData
- Microsoft.VisualBasic.CompilerServices.StandardModuleAttribute
- Microsoft.VisualBasic.CompilerServices.StaticLocalInitFlag
- Microsoft.VisualBasic.CompilerServices.Utils

Reflection Context (System.Reflection.Context namespace)

The System.Reflection.Context.CustomReflectionContext class isn't supported in .NET Native.

RTC (System.Net.Http.Rtc)

The `System.Net.Http.RtcRequestFactory` class isn't supported in .NET Native.

Windows Communication Foundation (WCF) (System.ServiceModel.*)

The types in the System.ServiceModel.* namespaces aren't supported in .NET Native. These include the following types:

- System.ServiceModel.ActionNotSupportedException
- System.ServiceModel.BasicHttpBinding
- System.ServiceModel.BasicHttpMessageCredentialType
- System.ServiceModel.BasicHttpSecurity
- System.ServiceModel.BasicHttpSecurityMode
- System.ServiceModel.CallbackBehaviorAttribute
- System.ServiceModel.ChannelFactory
- System.ServiceModel.ChannelFactory<TChannel>
- System.ServiceModel.ClientBase<TChannel>
- System.ServiceModel.ClientBase<TChannel>.BeginOperationDelegate
- System.ServiceModel.ClientBase<TChannel>.ChannelBase<T>
- System.ServiceModel.ClientBase<TChannel>.EndOperationDelegate
- System.ServiceModel.ClientBase<TChannel>.InvokeAsyncCompletedEventArgs
- System.ServiceModel.CommunicationException
- System.ServiceModel.CommunicationObjectAbortedException
- System.ServiceModel.CommunicationObjectFaultedException
- System.ServiceModel.CommunicationState
- System.ServiceModel.DataContractFormatAttribute

- System.ServiceModel.DnsEndpointIdentity
- System.ServiceModel.DuplexChannelFactory<TChannel>
- System.ServiceModel.DuplexClientBase<TChannel>
- System.ServiceModel.EndpointAddress
- System.ServiceModel.EndpointAddressBuilder
- System.ServiceModel.EndpointIdentity
- System.ServiceModel.EndpointNotFoundException
- System.ServiceModel.EnvelopeVersion
- System.ServiceModel.ExceptionDetail
- System.ServiceModel.FaultCode
- System.ServiceModel.FaultContractAttribute
- System.ServiceModel.FaultException
- System.ServiceModel.FaultException<TDetail>
- System.ServiceModel.FaultReason
- System.ServiceModel.FaultReasonText
- System.ServiceModel.HttpBindingBase
- System.ServiceModel.HttpClientCredentialType
- System.ServiceModel.HttpTransportSecurity
- System.ServiceModel.IClientChannel
- System.ServiceModel.ICommunicationObject
- System.ServiceModel.IContextChannel
- System.ServiceModel.IDefaultCommunicationTimeouts
- System.ServiceModel.IExtensibleObject<T>
- System.ServiceModel.IExtension<T>
- System.ServiceModel.IExtensionCollection<T>
- System.ServiceModel.InstanceContext
- System.ServiceModel.InvalidMessageContractException
- System.ServiceModel.MessageBodyMemberAttribute
- System.ServiceModel.MessageContractAttribute
- System.ServiceModel.MessageContractMemberAttribute
- System.ServiceModel.MessageCredentialType
- System.ServiceModel.MessageHeader<T>
- System.ServiceModel.MessageHeaderException
- System.ServiceModel.MessageParameterAttribute
- System.ServiceModel.MessageSecurityOverTcp
- System.ServiceModel.MessageSecurityVersion
- System.ServiceModel.NetHttpBinding
- System.ServiceModel.NetHttpMessageEncoding
- System.ServiceModel.NetTcpBinding
- System.ServiceModel.NetTcpSecurity

- System.ServiceModel.OperationContext
- System.ServiceModel.OperationContextScope
- System.ServiceModel.OperationContractAttribute
- System.ServiceModel.OperationFormatStyle
- System.ServiceModel.ProtocolException
- System.ServiceModel.QuotaExceededException
- System.ServiceModel.SecurityMode
- System.ServiceModel.ServerTooBusyException
- System.ServiceModel.ServiceActivationException
- System.ServiceModel.ServiceContractAttribute
- System.ServiceModel.ServiceKnownTypeAttribute
- System.ServiceModel.SpnEndpointIdentity
- System.ServiceModel.TcpClientCredentialType
- System.ServiceModel.TcpTransportSecurity
- System.ServiceModel.TransferMode
- System.ServiceModel.UnknownMessageReceivedEventArgs
- System.ServiceModel.UpnEndpointIdentity
- System.ServiceModel.XmlSerializerFormatAttribute
- System.ServiceModel.Channels.AddressHeader
- System.ServiceModel.Channels.AddressHeaderCollection
- System.ServiceModel.Channels.AddressingVersion
- System.ServiceModel.Channels.BinaryMessageEncodingBindingElement
- System.ServiceModel.Channels.ChannelManagerBase
- System.ServiceModel.Channels.ChannelParameterCollection
- System.ServiceModel.Channels.CommunicationObject
- System.ServiceModel.Channels.CompressionFormat
- System.ServiceModel.Channels.ConnectionOrientedTransportBindingElement
- System.ServiceModel.Channels.CustomBinding
- System.ServiceModel.Channels.FaultConverter
- System.ServiceModel.Channels.HttpRequestMessageProperty
- System.ServiceModel.Channels.HttpResponseMessageProperty
- System.ServiceModel.Channels.HttpsTransportBindingElement
- System.ServiceModel.Channels.HttpTransportBindingElement
- System.ServiceModel.Channels.IChannel
- System.ServiceModel.Channels.IChannelFactory
- System.ServiceModel.Channels.IChannelFactory<TChannel>
- System.ServiceModel.Channels.IDuplexChannel
- System.ServiceModel.Channels.IDuplexSession
- System.ServiceModel.Channels.IDuplexSessionChannel
- System.ServiceModel.Channels.IHttpCookieContainerManager

- System.ServiceModel.Channels.IInputChannel
- System.ServiceModel.Channels.IInputSession
- System.ServiceModel.Channels.IInputSessionChannel
- System.ServiceModel.Channels.IMessageProperty
- System.ServiceModel.Channels.IOutputChannel
- System.ServiceModel.Channels.IOutputSession
- System.ServiceModel.Channels.IOutputSessionChannel
- System.ServiceModel.Channels.IRequestChannel
- System.ServiceModel.Channels.IRequestSessionChannel
- System.ServiceModel.Channels.ISession
- System.ServiceModel.Channels.ISessionChannel<TSession>
- System.ServiceModel.Channels.LocalClientSecuritySettings
- System.ServiceModel.Channels.Message
- System.ServiceModel.Channels.MessageBuffer
- System.ServiceModel.Channels.MessageEncoder
- System.ServiceModel.Channels.MessageEncoderFactory
- System.ServiceModel.Channels.MessageEncodingBindingElement
- System.ServiceModel.Channels.MessageFault
- System.ServiceModel.Channels.MessageHeader
- System.ServiceModel.Channels.MessageHeaderInfo
- System.ServiceModel.Channels.MessageHeaders
- System.ServiceModel.Channels.MessageProperties
- System.ServiceModel.Channels.MessageState
- System.ServiceModel.Channels.MessageVersion
- System.ServiceModel.Channels.RequestContext
- System.ServiceModel.Channels.SecurityBindingElement
- System.ServiceModel.Channels.SecurityHeaderLayout
- System.ServiceModel.Channels.SslStreamSecurityBindingElement
- System.ServiceModel.Channels.TcpConnectionPoolSettings
- System.ServiceModel.Channels.TcpTransportBindingElement
- System.ServiceModel.Channels.TextMessageEncodingBindingElement
- System.ServiceModel.Channels.TransportBindingElement
- System.ServiceModel.Channels.TransportSecurityBindingElement
- System.ServiceModel.Channels.WebSocketTransportSettings
- System.ServiceModel.Channels.WebSocketTransportUsage
- System.ServiceModel.Channels.WindowsStreamSecurityBindingElement
- System.ServiceModel.Description.ClientCredentials
- System.ServiceModel.Description.ContractDescription
- System.ServiceModel.Description.DataContractSerializerOperationBehavior
- System.ServiceModel.Description.FaultDescription

- System.ServiceModel.Description.FaultDescriptionCollection
- System.ServiceModel.Description.IContractBehavior
- System.ServiceModel.Description.IEndpointBehavior
- System.ServiceModel.Description.IOperationBehavior
- System.ServiceModel.Description.MessageBodyDescription
- System.ServiceModel.Description.MessageDescription
- System.ServiceModel.Description.MessageDescriptionCollection
- System.ServiceModel.Description.MessageDirection
- System.ServiceModel.Description.MessageHeaderDescription
- System.ServiceModel.Description.MessageHeaderDescriptionCollection
- System.ServiceModel.Description.MessagePartDescription
- System.ServiceModel.Description.MessagePartDescriptionCollection
- System.ServiceModel.Description.MessagePropertyDescription
- System.ServiceModel.Description.MessagePropertyDescriptionCollection
- System.ServiceModel.Description.OperationDescription
- System.ServiceModel.Description.OperationDescriptionCollection
- System.ServiceModel.Description.ServiceEndpoint
- System.ServiceModel.Dispatcher.ClientOperation
- System.ServiceModel.Dispatcher.ClientRuntime
- System.ServiceModel.Dispatcher.DispatchOperation
- System.ServiceModel.Dispatcher.DispatchRuntime
- System.ServiceModel.Dispatcher.EndpointDispatcher
- System.ServiceModel.Dispatcher.IClientMessageFormatter
- System.ServiceModel.Dispatcher.IClientMessageInspector
- System.ServiceModel.Dispatcher.IClientOperationSelector
- System.ServiceModel.Dispatcher.IParameterInspector
- System.ServiceModel.Security.BasicSecurityProfileVersion
- System.ServiceModel.Security.HttpDigestClientCredential
- System.ServiceModel.Security.MessageSecurityException
- System.ServiceModel.Security.SecureConversationVersion
- System.ServiceModel.Security.SecurityAccessDeniedException
- System.ServiceModel.Security.SecurityPolicyVersion
- System.ServiceModel.Security.SecurityVersion
- System.ServiceModel.Security.TrustVersion
- System.ServiceModel.Security.UserNamePasswordClientCredential
- System.ServiceModel.Security.WindowsClientCredential
- System.ServiceModel.Security.Tokens.SecureConversationSecurityTokenParameters
- System.ServiceModel.Security.Tokens.SecurityTokenParameters
- System.ServiceModel.Security.Tokens.SupportingTokenParameters
- System.ServiceModel.Security.Tokens.UserNameSecurityTokenParameters

Differences in serializers

The following differences concern serialization and deserialization with the DataContractSerializer, DataContractJsonSerializer, and XmlSerializer classes:

- In .NET Native, DataContractSerializer and DataContractJsonSerializer fail to serialize or deserialize a derived class that has a base class member whose type isn't a root serialization type. For example, in the following code, trying to serialize or deserialize `Y` results in an error:

  C#

  ```
  using System;

  public class InnerType{}

  public class X
  {
     public InnerType instance { get; set; }
  }

  public class Y : X {}
  ```

  Type `InnerType` isn't known to the serializer, because the members of the base class aren't traversed during serialization.

- DataContractSerializer and DataContractJsonSerializer fail to serialize a class or structure that implements the IEnumerable<T> interface. For example, the following types fail to serialize or deserialize:

- XmlSerializer fails to serialize the following object value, because it doesn't know the exact type of the object to be serialized:

- XmlSerializer fails to serialize or deserialize if the type of the serialized object is XmlQualifiedName.

- All serializers (DataContractSerializer, DataContractJsonSerializer, and XmlSerializer) fail to generate serialization code for type System.Xml.Linq.XElement or for a type that contains XElement. They display build-time errors instead.

- The following constructors of the serialization types aren't guaranteed to work as expected:

  - DataContractSerializer(Type, IEnumerable<Type>)

  - DataContractSerializer(Type, DataContractSerializerSettings)

    - DataContractSerializer(Type, String, String, IEnumerable<Type>)
    - DataContractSerializer(Type, XmlDictionaryString, XmlDictionaryString, IEnumerable<Type>)
    - DataContractJsonSerializer(Type, DataContractJsonSerializerSettings)
    - DataContractJsonSerializer(Type, IEnumerable<Type>)
    - XmlSerializer(Type, String)
    - XmlSerializer(Type, Type[])
    - XmlSerializer(Type, XmlAttributeOverrides)
    - XmlSerializer(Type, XmlRootAttribute)
    - XmlSerializer(Type, XmlAttributeOverrides, Type[], XmlRootAttribute, String)
- XmlSerializer fails to generate code for a type that has methods attributed with any of the following attributes:
    - OnSerializingAttribute
    - OnSerializedAttribute
    - OnDeserializingAttribute
    - OnDeserializedAttribute
- XmlSerializer doesn't honor the IXmlSerializable custom serialization interface. If you have a class that implements this interface, XmlSerializer considers the type a plain old CLR object (POCO) type and serializes only its public properties.
- Serializing a plain Exception object doesn't work well with DataContractSerializer and DataContractJsonSerializer.

Visual Studio differences

Exceptions and debugging

When you're running apps compiled by using .NET Native in the debugger, first-chance exceptions are enabled for the following exception types:

- MemberAccessException
- TypeAccessException

Building apps

Use the x86 build tools that are used by default by Visual Studio. We don't recommend using the AMD64 MSBuild tools, which are found in C:\Program Files (x86)\MSBuild\12.0\bin\amd64; these may create build problems.

Profilers

- The Visual Studio CPU Profiler and XAML Memory Profiler don't display Just-My-Code correctly.
- The XAML Memory Profiler doesn't accurately display managed heap data.
- The CPU Profiler doesn't correctly identify modules, and displays prefixed function names.

Unit Test Library projects

Enabling .NET Native on a Unit Test Library for a Windows 8.x app project isn't supported and causes the project to fail to build.

See also

- Getting Started
- Runtime Directives (rd.xml) Configuration File Reference
- .NET Framework Support for Microsoft Store Apps and Windows Runtime

.NET Native general troubleshooting

Article • 10/20/2022

This article describes how to troubleshoot potential issues that you might encounter when developing apps with .NET Native.

Issues

- **Issue:** Your build output window isn't properly updated.

  Resolution: The build output window isn't updated until the build completes. Build times may be up to several minutes, so you might experience a delay in seeing the updates.

- **Issue:** Your app's retail build time for Arm has increased.

  Resolution: When you deploy an app to your Arm device, the .NET Native infrastructure is invoked. This compilation performs a large number of optimizations while ensuring that non-static semantics such as reflection continue to work. In addition, the portion of .NET Framework that the app uses is statically linked in for optimal performance and has to be compiled into native code as well. This is why compilation takes longer.

  However, compilation times are still within a minute of standard compilation for most apps on a standard development machine. Typically, just generating native images for .NET Framework on a standard development machine takes several minutes. Even with all the optimizations to improve the generated code and with including .NET Framework, app build times are typically a minute or two.

  We are continuing to work on improving compilation performance by investigating multi-threaded compilation and other optimizations.

- **Issue:** You don't know if your app was compiled using .NET Native.

  Resolution: If the .NET Native compiler is invoked, you'll notice longer build times, and Task Manager will show various .NET Native component processes such as ILC.exe and nutc_driver.exe.

  After you successfully build your project with .NET Native, you'll find the output under obj*config*\ *arch**projectname*.ilc\out. The final native package contents can be found under bin*arch**config*\AppX. The final native package contents are under \bin*arch**config*\AppX if you have deployed the app.

- **Issue:** Your .NET Native-compiled app is throwing runtime exceptions (typically MissingMetadataException or MissingRuntimeArtifactException exceptions) that it did not throw when compiled without .NET Native.

  Resolution: The exceptions are thrown because .NET Native did not provide either the metadata or the implementation code that is otherwise available through reflection. (For more information, see .NET Native and Compilation.) To eliminate the exception, you have to add an entry to your runtime directives (rd.xml) file so that the .NET Native tool chain can make the metadata or implementation code available at runtime. Two troubleshooters are available that will generate the necessary entry to add to your runtime directives file:

  - The MissingMetadataException troubleshooter for types.
  - The MissingMetadataException troubleshooter for methods.

  For more information, see Reflection and .NET Native.

See also

- Migrating Your Windows 8.x App to .NET Native

Security

Article • 07/11/2024

This section contains articles on building secure Universal Windows Platform (UWP) apps for Windows.

Introduction

If you're new to Windows or UWP development, start with the Intro to secure Windows app development. This introductory-level article provides an overview of security considerations for apps and the various features available in Windows.

Authentication and user identity

The authentication and user identity section contains walkthroughs for scenarios related to user login and identity. Apps have several options for user authentication, ranging from simple single sign-on (SSO) using Web authentication broker to highly secure two-factor authentication.

Expand table

Topic	Description
Credential locker	This article describes how apps can use the Credential Locker to securely store and retrieve user credentials, and roam them between devices with the user's Microsoft account.
Fingerprint biometrics	This article explains how to add fingerprint biometrics to your app. Including a request for fingerprint authentication when the user must consent to a particular action increases the security of your app. For example, you could require fingerprint authentication before authorizing an in-app purchase, or access to restricted resources. Fingerprint authentication is managed using the UserConsentVerifier class in the Windows.Security.Credentials.UI namespace.
Windows Hello	This article describes the Windows Hello technology, and discusses how developers can implement this technology to protect their apps and backend services. It highlights specific capabilities of these technologies that help mitigate threats from conventional credentials and provides guidance about designing and deploying these technologies as part of your packaged Windows apps.
Create a Windows Hello	Part 1 of a complete walkthrough on how to create a packaged Windows app that uses Windows Hello as an alternative to traditional username and

Topic	Description
login app	password authentication systems.
Create a Windows Hello login service	Part 2 of a complete walkthrough on how to use Windows Hello as an alternative to traditional username and password authentication systems in packaged Windows apps.
Smart cards	This topic explains how apps can use smart cards to connect users to secure network services, including how to access physical smart card readers, create virtual smart cards, communicate with smart cards, authenticate users, reset user PINs, and remove or disconnect smart cards.
Share certificates between apps	UWP apps that require secure authentication beyond a user Id and password combination can use certificates for authentication. Certificate authentication provides a high level of trust when authenticating a user. In some cases, a group of services will want to authenticate a user for multiple apps. This article shows how you can authenticate multiple apps using the same certificate, and how you can provide convenient code for a user to import a certificate that was provided to access secured web services.
Windows Unlock with companion IoT devices	A companion device is a device that can act in conjunction with Windows to enhance the user authentication experience. Using the Companion Device Framework, a companion device can provide a rich experience even when Windows Hello is not available (for example, if the Windows machine lacks a camera for face authentication or fingerprint reader device, for example).
Web Account Manager	This article describes how to show the AccountsSettingsPane and connect your Universal Windows Platform (UWP) app to external identity providers, like Microsoft or Facebook, using the Windows Web Account Manager APIs. You'll learn how to request a user's permission to use their Microsoft account, obtain an access token, and use it to perform basic operations (like get profile data or upload files to their OneDrive).
Web authentication broker	This article explains how to connect your app to an online identity provider that uses authentication protocols like OpenID or OAuth. The AuthenticateAsync method sends a request to the online identity provider and gets back an access token that describes the provider resources to which the app has access.

Cryptography

The cryptography section contains information on more complex, cryptographic related topics.

Expand table

Topic	Description
Intro to certificates	This article discusses the use of certificates in apps. Digital certificates are used in public key cryptography to bind a public key to a person, computer, or organization. The bound identities are most often used to authenticate one entity to another. For example, certificates are often used to authenticate a web server to a user and a user to a web server. You can create certificate requests and install or import issued certificates. You can also enroll a certificate in a certificate hierarchy.
Cryptographic keys	This article shows how to use standard key derivation functions to derive keys and how to encrypt content using symmetric and asymmetric keys.
Data protection	This article explains how to use the DataProtectionProvider class in the Windows.Security.Cryptography.DataProtection namespace to encrypt and decrypt digital data in a UWP app.
MACs, hashes, and signatures	This article discusses how message authentication codes (MACs), hashes, and signatures can be used in apps to detect message tampering.
Export restrictions on cryptography	Use this info to determine if your app uses cryptography in a way that might prevent it from being listed in the Microsoft Store.
Common cryptography tasks	These articles provide example code for common cryptography tasks, such as creating random numbers, comparing buffers, converting between strings and binary data, copying to and from byte arrays, and encoding and decoding data.

Feedback

Intro to secure Windows app development

Article • 07/08/2024

This introductory article helps app architects and developers better understand the various Windows 10 platform capabilities that accelerate creating secure Universal Windows Platform (UWP) apps. It details how to use the Windows security features available at each of the following stages: authentication, data-in-flight, and data-at-rest. You can find more in-depth information on each topic by reviewing the additional resources included in each chapter.

1 Introduction

Developing a secure app can be a challenge. In today's fast-paced world of mobile, social, cloud, and complex enterprise apps, customers expect apps to become available and updated faster than ever. They also use many types of devices, further adding to the complexity of creating app experiences. If you build for the Windows Universal Windows Platform (UWP), that could include the traditional list of desktops, laptops, tablets, and mobile devices; in addition to a growing list of new devices spanning the Internet of Things, Xbox One, Microsoft Surface Hub, and HoloLens. As the developer, you must ensure your apps communicate and store data securely, across all the platforms or devices involved.

Here are some of the benefits of utilizing Windows security features.

- You will have standardized security across all devices that support Windows, by using consistent APIs for security components and technologies.
- You write, test, and maintain less code than you would if you implemented custom code to cover these security scenarios.
- Your apps become more stable and secure because you use the operating system to control how the app accesses its resources and local or remote system resources.

During authentication, the identity of a user requesting access to a particular service is validated. Windows Hello is the component in Windows that helps create a more secure authentication mechanism in Windows apps. With it, you can use a Personal Identification Number (PIN) or biometrics such as the user's fingerprints, face, or iris to implement multi-factor authentication for your apps.

Data-in-flight refers to the connection and the messages transferred across it. An example of this is retrieving data from a remote server using web services. The use of Secure Sockets Layer (SSL) and Secure Hypertext Transfer Protocol (HTTPS) ensures the security of the connection. Preventing intermediary parties from accessing these messages, or unauthorized apps from communicating with the web services, is key to securing data in flight.

Lastly, data-at-rest relates to data residing in memory or on storage media. Windows apps have an app model that prevents unauthorized data access between apps, and offers encryption APIs to further secure data on the device. A feature called Credential Locker can be used to securely store user credentials on the device, with the operating system preventing other apps from accessing them.

2 Authentication Factors

To protect data, the person requesting access to it must be identified and authorized to access the data resources they request. The process of identifying a user is called authentication, and determining access privileges to a resource is called authorization. These are closely related operations, and to the user they might be indistinguishable. They can be relatively simple or complex operations, depending on many factors: for example, whether the data resides on one server or is distributed across many systems. The server providing the authentication and authorization services is referred to as the identity provider.

To authenticate themselves with a particular service and/or app, the user employs credentials made up of something they know, something they have, and/or something they are. Each of these are called authentication factors.

- **Something the user knows** is usually a password, but it can also be a personal identification number (PIN) or a “secret” question-and-answer pair.
- **Something the user has** is most often a hardware memory device such as a USB stick containing the authentication data unique to the user.
- **Something the user is** often encompasses their fingerprints, but there are increasingly popular factors like the user’s speech, facial, ocular (eye) characteristics, or patterns of behavior. When stored as data, these measurements are called biometrics.

A password created by the user is an authentication factor in itself, but it often isn’t sufficient; anyone who knows the password can impersonate the user who owns it. A smart card can provide a higher level of security, but it might be stolen, lost, or misplaced. A system that can authenticate a user by their fingerprint or by an ocular scan might provide the highest and most convenient level of security, but it requires

expensive and specialized hardware (for example, an Intel RealSense camera for facial recognition) that might not be available to all users.

Designing the method of authentication used by a system is a complex and important aspect of data security. In general, the greater number of factors you use in authentication, the more secure the system is. At the same time, authentication must be usable. A user will usually log in many times a day, so the process must be fast. Your choice of authentication type is a trade-off between security and ease of use; single-factor authentication is the least secure and easiest to use, and multi-factor authentication becomes more secure, but more complex as more factors are added.

2.1 Single-factor authentication

This form of authentication is based on a single user credential. This is usually a password, but it could also be a personal identification number (PIN).

Here's the process of single-factor authentication.

- The user provides their username and password to the identity provider. The identity provider is the server process that verifies the identity of the user.
- The identity provider checks whether the username and password are the same as those stored in the system. In most cases, the password will be encrypted, providing additional security so that others cannot read it.
- The identity provider returns an authentication status that indicates whether the authentication was successful.
- If successful, data exchange begins. If unsuccessful, the user must be re-authenticated.

Today, this method of authentication is the most commonly used one across services. It is also the least secure form of authentication when used as the only means of authentication. Password complexity requirements, "secret questions," and regular password changes can make using passwords more secure, but they put more burden on users and they're not an effective deterrent against hackers.

The challenge with passwords is that it is easier to guess them successfully than systems that have more than one factor. If they steal a database with user accounts and hashed password from a little web shop, they can use the passwords used on other web sites.

Users tend to reuse accounts all the time, because complex passwords are hard to remember. For an IT department, managing passwords also brings with it the complexity of having to offer reset mechanisms, requiring frequent updates to passwords, and storing them in a safe manner.

For all of its disadvantages, single-factor authentication gives the user control of the credential. They create it and modify it, and only a keyboard is needed for the authentication process. This is the main aspect that distinguishes single-factor from multi-factor authentication.

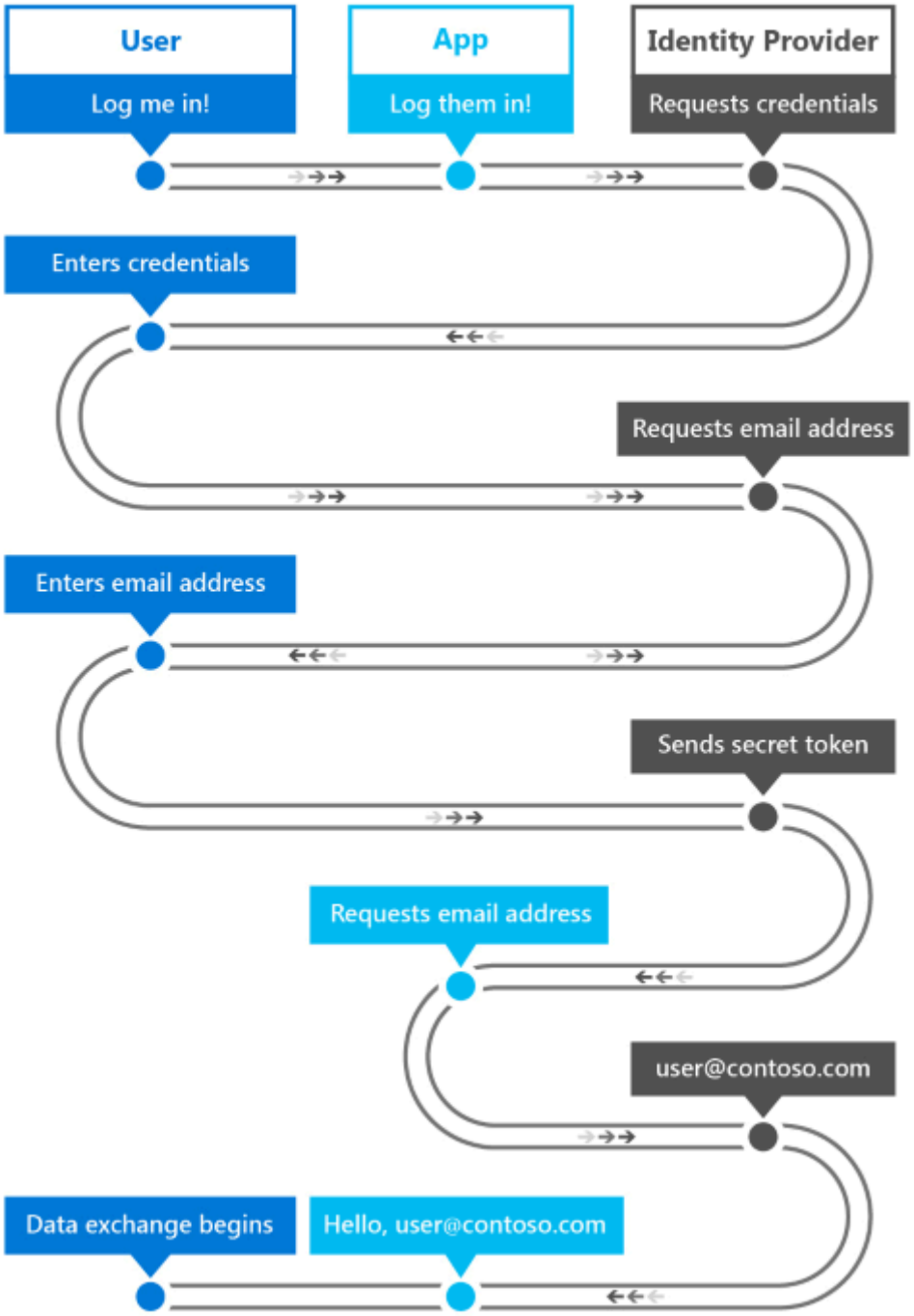
2.1.1 Web authentication broker

As previously discussed, one of the challenges with password authentication for an IT department is the added overhead of managing the base of usernames/passwords, reset mechanisms, etc. An increasingly popular option is to rely on third-party identity providers that offer authentication through OAuth, an open standard for authentication.

Using OAuth, IT departments can effectively "outsource" the complexity of maintaining a database with usernames and passwords, reset password functionality, etc. to a third party identity provider like Facebook, X or Microsoft.

Users have complete control over their identity on these platforms, but apps can request a token from the provider, after the user is authenticated and with their consent, which can be used to authorize authenticated users.

The web authentication broker in Windows provides a set of APIs and infrastructure for apps to use authentication and authorization protocols like OAuth and OpenID. Apps can initiate authentication operations through the **WebAuthenticationBroker** API, resulting in the return of a **WebAuthenticationResult**. An overview of the communication flow is illustrated in the following figure.



The app acts as the broker, initiating the authentication with the identity provider through a **WebView** in the app. When the identity provider has authenticated the user, it returns a token to the app that can be used to request information about the user from the identity provider. As a security measure, the app must be registered with the identity provider before it can broker the authentication processes with the identity provider. This registration steps differ for each provider.

Here's the general workflow for calling the **WebAuthenticationBroker** API to communicate with the provider.

- Construct the request strings to be sent to the identity provider. The number of strings, and the information in each string, is different for each web service but it usually includes two URI strings each containing a URL: one to which the

authentication request is sent, and one to which the user is redirected after authorization is complete.
- Call **WebAuthenticationBroker.AuthenticateAsync**, passing in the request strings, and wait for the response from the identity provider.
- Call **WebAuthenticationResult.ResponseStatus** to get the status when the response is received.
- If the communication is successful, process the response string returned by the identity provider. If unsuccessful, process the error.

If the communication is successful, process the response string returned by the identity provider. If unsuccessful, process the error.

Sample C# code that for this process is below. For information and a detailed walkthrough, see WebAuthenticationBroker. For a complete code sample, check out the WebAuthenticationBroker sample on GitHub.

cs

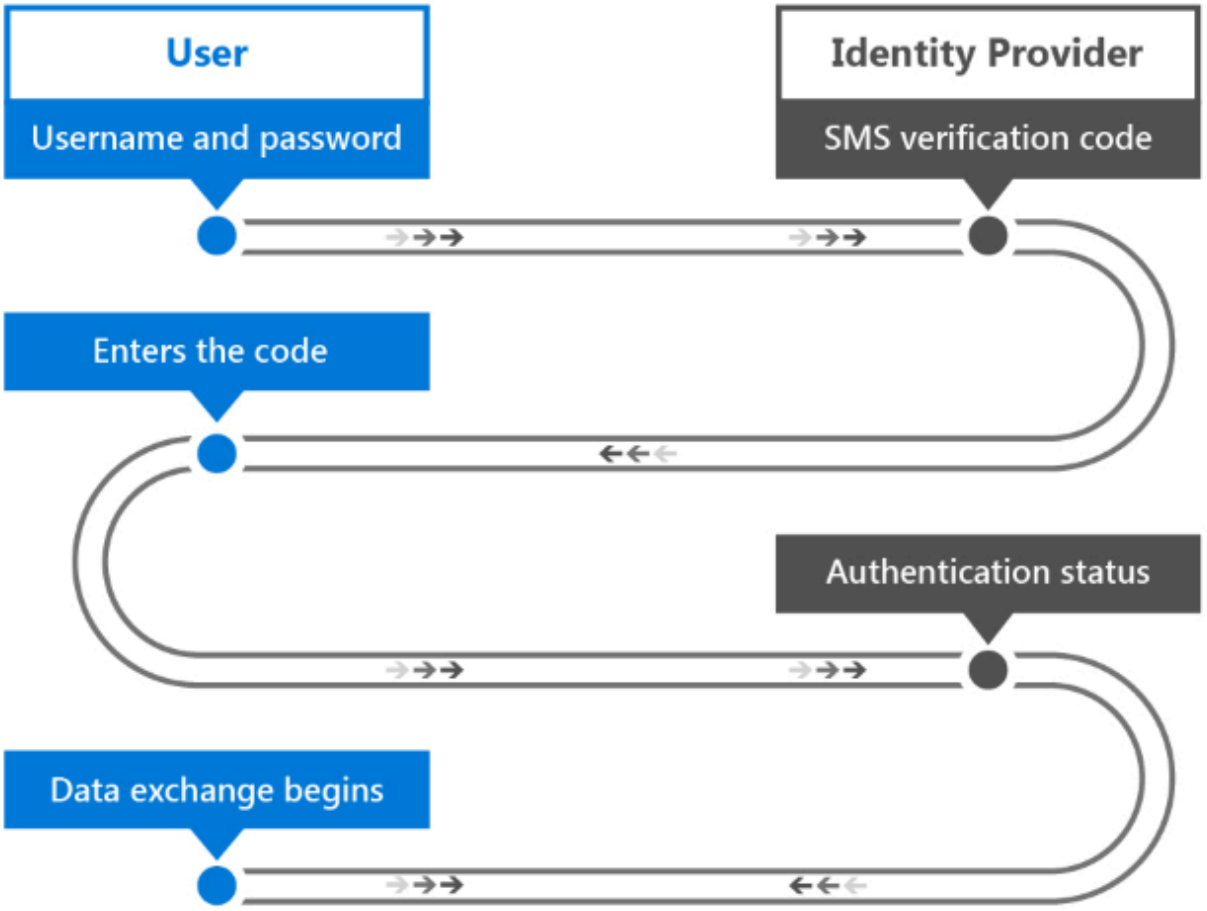
```cs
string startURL = "https://<providerendpoint>?client_id=<clientid>";
string endURL = "http://<AppEndPoint>";

var startURI = new System.Uri(startURL);
var endURI = new System.Uri(endURL);

try
{
    WebAuthenticationResult webAuthenticationResult =
        await WebAuthenticationBroker.AuthenticateAsync(
            WebAuthenticationOptions.None, startURI, endURI);

    switch (webAuthenticationResult.ResponseStatus)
    {
        case WebAuthenticationStatus.Success:
            // Successful authentication.
            break;
        case WebAuthenticationStatus.ErrorHttp:
            // HTTP error.
            break;
        default:
            // Other error.
        break;
    }
}
catch (Exception ex)
{
    // Authentication failed. Handle parameter, SSL/TLS, and
    // Network Unavailable errors here.
}
```

2.2 Multi-factor authentication

Multi-factor authentication makes use of more than one authentication factor. Usually, "something you know," such as a password, is combined with "something you have," which can be a mobile phone or a smart card. Even if an attacker discovers the user's password, the account is still inaccessible without the device or card. And if only the device or card is compromised, it is not useful to the attacker without the password. Multi-factor authentication is therefore more secure, but also more complex, than single-factor authentication.

Services that use multi-factor authentication will often give the user a choice in how they receive the second credential. An example of this type of authentication is a commonly used process where a verification code is sent to the user's mobile phone using SMS.

- The user provides their username and password to the identity provider.
- The identity provider verifies the username and password as in single-factor authorization, and then looks up the user's mobile phone number stored in the system.
- The server sends an SMS message containing a generated verification code to the user's mobile phone.
- The user provides the verification code to the identity provider; through a form presented to the user.
- The identity provider returns an authentication status that indicates whether the authentication of both credentials were successful.
- If successful, data exchange begins. Otherwise, the user must be re-authenticated.



As you can see, this process also differs from single-factor authentication in that the second user credential is sent to the user instead of being created or provided by the user. The user is therefore not in complete control of the necessary credentials. This also applies when a smart card is used as the second credential: the organization is in charge of creating and providing it to the user.

2.2.1 Azure Active Directory

Azure Active Directory (Azure AD) is a cloud-based identity and access management service that can serve as the identity provider in single-factor or multi-factor authentication. Azure AD authentication can be used with or without a verification code.

While Azure AD can also implement single-factor authentication, enterprises usually require the higher security of multi-factor authentication. In a multi-factor authentication configuration, a user authenticating with an Azure AD account has the option of having a verification code sent as an SMS message either to their mobile phone or the Azure Authenticator mobile app.

Additionally, Azure AD can be used as an OAuth provider, providing the standard user with an authentication and authorization mechanism to apps across various platforms. To learn more, see Azure Active Directory and Multi-Factor Authentication on Azure.

2.4 Windows Hello

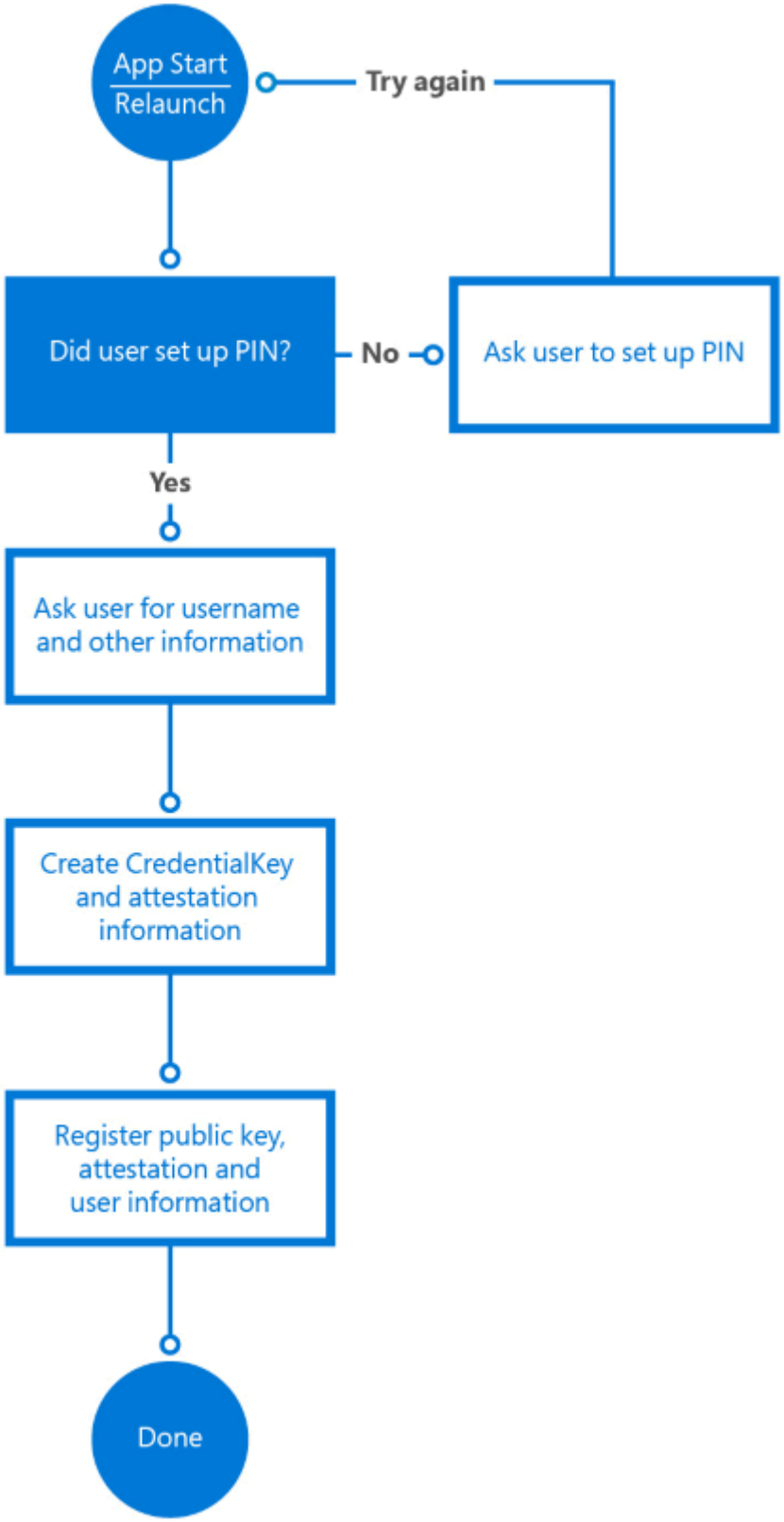
In Windows, a convenient multi-factor authentication mechanism is built into the operating system. Windows Hello is the new biometric sign-in system built into Windows. Because it is built directly into the operating system, Windows Hello allows face or fingerprint identification to unlock users' devices. The Windows secure credential store protects biometric data on the device.

Windows Hello provides a robust way for a device to recognize an individual user, which addresses the first part of the path between a user and a requested service or data item. After the device has recognized the user, it still must authenticate the user before determining whether to grant access to a requested resource. Windows Hello also provides strong two-factor authentication (2FA) that is fully integrated into Windows and replaces reusable passwords with the combination of a specific device, and a biometric gesture or PIN. The PIN is specified by the user as part of their Microsoft account enrollment.

Windows Hello isn't just a replacement for traditional 2FA systems, though. It's conceptually similar to smart cards: authentication is performed by using cryptographic primitives instead of string comparisons, and the user's key material is secure inside tamper-resistant hardware. Microsoft Hello doesn't require the extra infrastructure components required for smart card deployment, either. In particular, you don't need a Public Key Infrastructure (PKI) to manage certificates, if you don't currently have one. Windows Hello combines the major advantages of smart cards—deployment flexibility for virtual smart cards and robust security for physical smart cards—without any of their drawbacks.

A device must be registered with Windows Hello before users can authenticate with it. Windows Hello uses asymmetric (public/private key) encryption in which one party uses a public key to encrypt the data that the other party can decrypt using a private key. In the case of Windows Hello, it creates a set of public/private key pairs and writes the private keys to the device's Trusted Platform Module (TPM) chip. After a device has been registered, UWP apps can call system APIs to retrieve the user's public key, which can be used to register the user on the server.

The registration workflow of an app might look like the following:



The registration information you collect may include a lot more identifying information than it does in this simple scenario. For example, if your app accesses a secured service such as one for banking, you'd need to request proof of identity and other things as part of the sign-up process. Once all the conditions are met, the public key of this user will be stored in the back-end and used to validate the next time the user uses the service.

For more information on Windows Hello, see the Windows Hello for Business overview and the Windows Hello developer guide.

3 Data-in-flight security methods

Data-in-flight security methods apply to data in transit between devices connected to a network. The data may be transferred between systems on the high-security environment of a private corporate intranet, or between a client and web service in the non-secure environment of the web. Windows apps support standards such as SSL through their networking APIs, and work with technologies such as Azure API Management with which developers can ensure the appropriate level of security for their apps.

3.1 Remote system authentication

There are two general scenarios where communication occurs with a remote computer system.

- A local server authenticates a user over a direct connection. For example, when the server and the client are on a corporate intranet.
- A web service is communicated with over the Internet.

Security requirements for web service communication are higher than those in direct connection scenarios, as data is no longer only a part of a secure network and the likelihood of malicious attackers looking to intercept data is also higher. Because various types of devices will access the service, they will likely be built as RESTful services, as opposed to WCF, for instance, which means authentication and authorization to the service also introduces new challenges. We'll discuss two requirements for secure remote system communication.

The first requirement is message confidentiality: The information passed between the client and the web services (for example, the identity of the user and other personal information) must not be readable by third parties while in transit. This is usually accomplished by encrypting the connection over which messages are sent and by encrypting the message itself. In private/public key encryption, the public key is available to anyone, and is used to encrypt messages to be sent to a specific receiver. The private key is only held by the receiver and is used to decrypt the message.

The second requirement is message integrity: The client and the web service must be able to verify that the messages they receive are the ones intended to be sent by the

other party, and that the message has not been altered in transit. This is accomplished by signing messages with digital signatures and using certificate authentication.

3.2 SSL connections

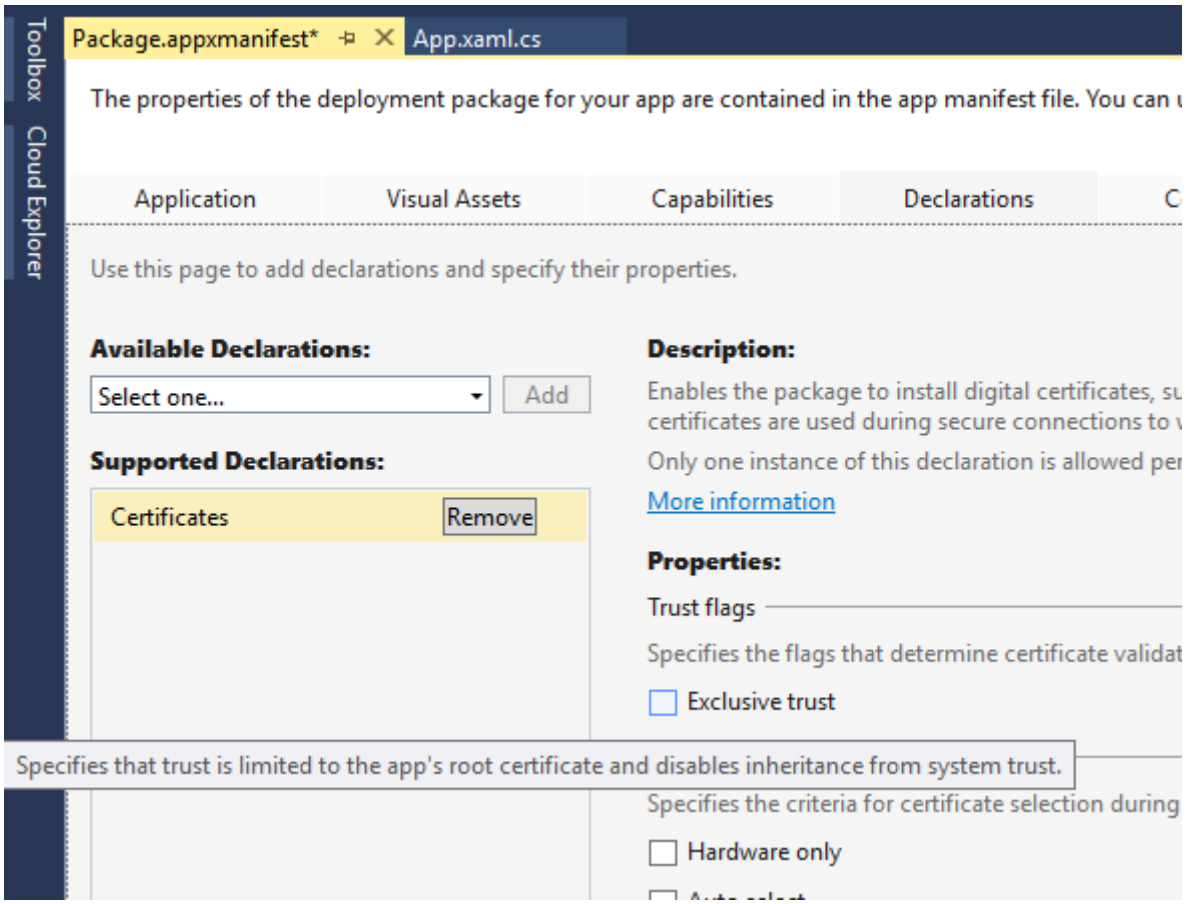
To establish and maintain secure connections to clients, web services can use Secure Sockets Layer (SSL), which is supported by the Secure Hypertext Transfer Protocol (HTTPS). SSL provides message confidentiality and integrity by supporting public key encryption as well as server certificates. SSL is superseded by Transport Layer Security (TLS), but TLS is often casually referred to as SSL.

When a client requests access to a resource on a server, SSL starts a negotiation process with the server. This is called an SSL handshake. An encryption level, a set of public and private encryption keys, and the identity information in the client and server certificates are agreed upon as the basis of all communication for the duration of the SSL connection. The server may also require the client to be authenticated at this time. Once the connection is established, all messages are encrypted with the negotiated public key until the connection closes.

3.2.1 SSL pinning

While SSL can provide message confidentiality using encryption and certificates, it does nothing to verify that the server with which the client is communicating is the correct one. The server's behavior can be mimicked by an unauthorized third-party, intercepting the sensitive data that the client transmits. To prevent this, a technique called SSL pinning is used to verify that the certificate on the server is the certificate that the client expects and trusts.

There are a few different ways to implement SSL pinning in apps, each with their own pros and cons. The easiest approach is via the Certificates declaration in the app's package manifest. This declaration enables the app package to install digital certificates and specify exclusive trust in them. This results in SSL connections being allowed only between the app and servers that have the corresponding certificates in their certificate chain. This mechanism also enables the secure use of self-signed certificates, as no third party dependency is needed on trusted public certification authorities.



For more control over the validation logic, APIs are available to validate the certificate(s) returned by the server in response to an HTTPS request. Note that this method requires sending a request and inspecting the response, so be sure to add this as a validation before actually sending sensitive information in a request.

The following C# code illustrates this method of SSL pinning. The **ValidateSSLRoot** method uses the **HttpClient** class to execute an HTTP request. After the client sends the response, it uses the **RequestMessage.TransportInformation.ServerIntermediateCertificates** collection to inspect the certificates returned by the server. The client can then validate the entire certificate chain with the thumbprints it has included. This method does require the certificate thumbprints to be updated in the app when the server certificate expires and is renewed.

cs

```
private async Task ValidateSSLRoot()
{
    // Send a get request to Bing
    var httpClient = new HttpClient();
    var bingUri = new Uri("https://www.bing.com");
    HttpResponseMessage response =
        await httpClient.GetAsync(bingUri);

    // Get the list of certificates that were used to
    // validate the server's identity
```

```
    IReadOnlyList<Certificate> serverCertificates =
response.RequestMessage.TransportInformation.ServerIntermediateCertificates;

    // Perform validation
    if (!ValidateCertificates(serverCertificates))
    {
        // Close connection as chain is not valid
        return;
    }
    // Validation passed, continue with connection to service
}

private bool ValidateCertificates(IReadOnlyList<Certificate> certs)
{
    // In this example, we iterate through the certificates
    // and check that the chain contains
    // one specific certificate we are expecting
    foreach (var cert in certs)
    {
        byte[] thumbprint = cert.GetHashValue();

        // Check if the thumbprint matches whatever you
        // are expecting
        var expected = new byte[] { 212, 222, 32, 208, 94, 102,
            252, 83, 254, 26, 80, 136, 44, 120, 219, 40, 82, 202,
            228, 116 };

        // ThumbprintMatches does the byte[] comparison
        if (ThumbprintMatches(thumbprint, expected))
        {
            return true;
        }
    }
    return false;
}
```

3.3 Publishing and securing access to REST APIs

To ensure authorized access to web services, they must require authentication every time an API call is made. Being able to control performance and scale is also something to consider when web services are exposed across the web. Azure API Management is a service that can help expose APIs across the web, while providing features on three levels.

Publishers/Administrators of the API can easily configure the API through the Publisher Portal of Azure API Management. Here, API sets can be created and access to them can be managed to control who has access to which APIs.